VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

# Quiz Review: Instruction Sets

## Advanced System Architectures

**CO5059**

**Instructor:** Tran Ngoc Thinh

**Notes by:** Phan Nhat Minh (minh.phannhat95@hcmut.edu.vn)

**Semester:** 261

*Lecture 2 – ISA styles, addressing modes, CISC vs. RISC, MIPS encoding, procedure calls. A cheat sheet, the slide exercises with solutions, exam-style questions, and extra practice problems – with solutions at the end.*

Last updated: October 4, 2026

# Contents

**1 Cheat Sheet** **1**
- 1.1 ISA styles: $C = A + B$ . . . 1
- 1.2 Addressing modes . . . 1
- 1.3 MIPS instruction formats . . . 1
- 1.4 Encoding recipe . . . 1
- 1.5 Registers and calling convention . . . 2
- 1.6 Pseudo-instructions and tricks . . . 2
- 1.7 CISC vs. RISC . . . 2

**2 Exercises from the Slides (with Solutions)** **2**

**3 Exam-Style Questions** **5**

**4 Practice Problems** **6**

**5 Solutions to the Practice Problems** **7**

**6 Solutions to the Exam-Style Questions** **9**

# 1 Cheat Sheet

## 1.1 ISA styles: $C = A + B$

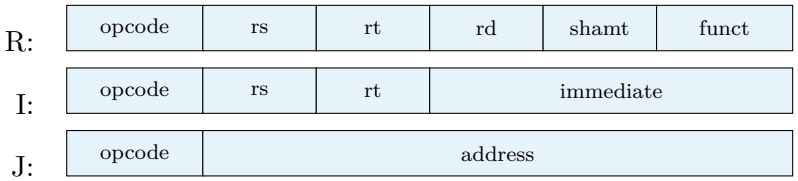
| Style | Code | Pros / cons |
|---|---|---|
| Stack | `PUSH A; PUSH B; ADD; POP C` | + dense code, simple HW; – stack is a bottleneck, little parallelism |
| Accumulator | `LOAD A; ADD B; STORE C` | + few instructions, easy compiler; – heavy memory traffic, variable CPI |
| Memory-memory | `ADD C, A, B` | + simple to understand; – every operand is a memory access |
| Register-memory | `LOAD R1,A; ADD R3,R1,B; STORE R3,C` | one operand may be in memory (x86, 68000, VAX) |
| Load-store (reg-reg) | `LOAD R1,A; LOAD R2,B; ADD R3,R1,R2; STORE R3,C` | only loads/stores touch memory (MIPS, ARM, PowerPC, Alpha) |

## 1.2 Addressing modes

| Mode | Meaning | MIPS example |
|---|---|---|
| Immediate | operand is in the instruction | `addi $t0,$t1,5` |
| Register | operand is in a register | `add $t0,$t1,$t2` |
| Displacement (base) | $M$[reg + const] | `lw $t0,8($sp)` |
| Register indirect | $M$[reg] (or PC = reg) | `lw $t0,0($a0)`, `jr $ra` |
| PC-relative | target = PC+4 + 4·offset | `beq $t0,$t1,L` |
| Pseudo-direct | target = {PC+4[31:28], addr, 00} | `j L`, `jal f` |
| Direct, indexed, memory indirect, scaled | not in MIPS | e.g. `M[M[r2]]` |

Measured usage: **immediate and displacement dominate**; most displacements and immediates fit in 16 bits.

## 1.3 MIPS instruction formats




Bit positions: opcode 31–26, rs 25–21, rt 20–16, rd 15–11, shamt 10–6, funct 5–0, immediate 15–0, address 25–0. In I-format `rt` is the *destination* (or the value stored by `sw`).

## 1.4 Encoding recipe

1. Look up opcode (and funct for R-format; opcode is `0`).
2. Write register numbers (5 bits each) in the order **rs, rt, rd**. Note: in assembly `add rd, rs, rt` lists rd first.
3. Immediates: 16-bit two's complement (negative: invert and add 1).
4. Branch offset = (target − (PC + 4))/4 in words. Jump field = (target ≫ 2) & `0x3FFFFFF`.
5. Group the 32 bits into nibbles → hex.

| Instr. | opcode / funct (hex) | Instr. | opcode / funct (hex) |
|---|---|---|---|
| `add` | 0 / 20 | `addi` | 08 |
| `addu` | 0 / 21 | `addiu` | 09 |
| `sub` | 0 / 22 | `andi` | 0c |
| `and` | 0 / 24 | `ori` | 0d |
| `or` | 0 / 25 | `slti` | 0a |
| `nor` | 0 / 27 | `lui` | 0f |
| `slt` | 0 / 2a | `lw` | 23 |
| `sll` | 0 / 00 | `sw` | 2b |
| `srl` | 0 / 02 | `beq` | 04 |
| `sra` | 0 / 03 | `bne` | 05 |
| `jr` | 0 / 08 | `j` / `jal` | 02 / 03 |

## 1.5 Registers and calling convention

| Name | Number | Use | Preserved? |
|---|---|---|---|
| `$zero` | 0 | constant 0 | – |
| `$at` | 1 | assembler temporary | – |
| `$v0--$v1` | 2–3 | return values | no |
| `$a0--$a3` | 4–7 | arguments | no |
| `$t0--$t7`, `$t8--$t9` | 8–15, 24–25 | temporaries (caller-saved) | no |
| `$s0--$s7` | 16–23 | saved (callee-saved) | yes |
| `$gp, $sp, $fp, $ra` | 28, 29, 30, 31 | global/stack/frame pointer, return address | yes |

**Note.** `jal f` puts PC+4 into `$ra`; `jr $ra` returns. A callee that uses `$s` registers or calls another function must save them (and `$ra`) on the stack; a *leaf* function that does not touch `$ra`/`$s` registers needs no stack frame.

## 1.6 Pseudo-instructions and tricks

| Pseudo / idea | Real instructions |
|---|---|
| `move $t1,$t0` | `add $t1,$t0,$zero` |
| `li $t0,100` (fits 16 bits) | `ori $t0,$zero,100` (or `addi`) |
| `li $t0,0x12345678` | `lui $t0,0x1234; ori $t0,$t0,0x5678` |
| `not $t0,$t1` | `nor $t0,$t1,$zero` |
| `blt $t0,$t1,L` | `slt $at,$t0,$t1; bne $at,$zero,L` |
| `bge $t0,$t1,L` | `slt $at,$t0,$t1; beq $at,$zero,L` |
| `subi` (doesn't exist) | `addi` with a negative constant |

## 1.7 CISC vs. RISC

| CISC (x86, 680x0, VAX) | RISC (MIPS, SPARC, PowerPC, Alpha) |
|---|---|
| More work per instruction, fewer instructions | Simple instructions, more of them |
| Motivated by expensive memory | Designed for pipelining, target CPI $\leq 1$ |
| Variable-length encoding | Fixed-length (32-bit) encoding |
| Many addressing modes (12–25) | Few modes (immediate, register, displacement, …) |
| Memory operands in ALU instructions | Load-store: only `lw`/`sw` access memory |

# 2 Exercises from the Slides (with Solutions)

**Exercise 2.1** (Encode an R-format instruction)**.** Encode `add $t0, $s1, $s2`.

**Solution.** opcode 0, rs = `$s1` = 17, rt = `$s2` = 18, rd = `$t0` = 8, shamt 0, funct `0x20`:

`000000 10001 10010 01000 00000 100000` = `0x02324020`

□

**Exercise 2.2** (Shifts)**.** Encode `sll $t2,$s0,8` and `sra $t2,$s0,8`. Why is there no `sla`, and how is `NOT` done?

**Solution.** Shifts use rt as source and shamt as the amount; rs = 0. `sll`:

`000000 00000 10000 01010 01000 000000` = `0x00105200`

`sra`: same with funct `000011` = `0x00105203`. A left arithmetic shift would be identical to `sll` (zeros enter from the right either way), so it is not needed. `NOT` is `nor $t0,$t1,$zero`.

□

**Exercise 2.3** (I-format arithmetic and logical)**.** Encode `addi $t0,$t1,0x0005` and `ori $t0,$t1,0xFF00`. Why is there no `subi`?

**Solution.** rs = `$t1` = 9, rt = `$t0` = 8. `addi`:

`001000 01001 01000 0000000000000101` = `0x21280005`

`ori`:

`001101 01001 01000 1111111100000000` = `0x3528FF00`

`subi` is unnecessary: `addi` with a negative (sign-extended) immediate does the same job.

□

**Exercise 2.4** (Loads and stores)**.** Encode `lw $t0,16($t1)` and `sw $t0,16($t1)`.

**Solution.** rs = base `$t1` = 9, rt = `$t0` = 8, imm = 16. `lw`:

`100011 01001 01000 0000000000010000` = `0x8D280010`

`sw`: opcode `101011` ⇒ `0xAD280010`. □

**Exercise 2.5** (Branch)**.** Encode `beq $s0,$s1,256` (offset field = 256).

**Solution.** Encoding:

`000100 10000 10001 0000000100000000` = `0x12110100`

The target is PC+4 + 256 × 4 = PC+4 + 1024. □

**Exercise 2.6** (Jump). Encode `j 0x01000000` as shown on the slide.

**Solution.** The slide places `0x01000000` directly in the 26-bit field:

$$\texttt{000010}\ \ \texttt{01000000000000000000000000} = \texttt{0x09000000}$$

The real target is {PC+4[31:28], field, 00}, i.e. the field is a *word* address. If you are given a byte target address instead, divide it by 4 first (see Practice Problem 4.6). □

**Exercise 2.7** (Move without MOV). Move `$t0` into `$t1` with one instruction.

**Solution.** `add $t1,$t0,$zero` (`or $t1,$t0,$zero` or `addi $t1,$t0,0` also work). □

**Exercise 2.8** (Large constant). Load `0x12345678` into `$t0`.

**Solution.** `lui $t0,0x1234` then `ori $t0,$t0,0x5678`. `ori` zero-extends its immediate, so the upper half is not disturbed. □

**Exercise 2.9** (Atomic swap with LL/SC). Explain the loop `try: add $t0,$zero,$s4; ll $t1,0($s1); sc $t0,0($s1); beq $t0,$zero,try; add $s4,$zero,$t1`.

**Solution.** `ll` reads the old memory value into `$t1`. `sc` tries to store `$s4`'s copy; it writes 1 into `$t0` on success, or 0 if the location changed since the `ll` (another processor intervened), in which case the loop retries. On success `$s4` receives the old value: memory and `$s4` have been swapped atomically. □

**Exercise 2.10** (Procedure-call convention). (1) Who saves the return address, caller or callee? (2) Who saves the necessary registers? (3) Must a leaf procedure save the return address?

**Solution.** (1) The caller's `jal` writes the return address into `$ra`; it is the **callee** that must preserve it (push `$ra` on the stack) if the callee itself calls other procedures, because `$ra` is "preserved across a call". (2) Both share the work: the **callee** saves the `$s` registers it uses (callee-saved); the **caller** saves any `$t`/`$a`/`$v` registers it still needs after the call (caller-saved). (3) **No**: a leaf procedure makes no `jal`, so `$ra` is never overwritten and it can return with `jr $ra` directly. □

**Exercise 2.11** (Review quiz). (1) In MIPS, `NOT` is done with: a. `NOT`, b. `NOR`, c. `NAND`, d. none. (2) An addressing mode is: a. how an operand (or a jump target) is referenced, b. how memory is organized by byte/halfword/word, c. endianness, d. all of these. (3) MIPS `LO`/`HI` are used for: a. low/high addresses, b. multiply/divide, c. both, d. neither. (4) MIPS stores data as: a. little-endian, b. big-endian, c. both, d. no endianness.

**Solution.** (1) **b** – `nor $t0,$t1,$zero`. (2) **a**. (3) **b** – `mult`/`div` put results in Hi/Lo (`mfhi`/`mflo` read them). (4) **b** – big-endian, as stated in the lecture (real MIPS chips can be configured either way, but the expected quiz answer is b). □

**Exercise 2.12** (C to MIPS). Which sequences implement `if (a==0) a=1; else a=0;`?

(A)
```
lw $t0,a; bne $t0,$0,L1; addi $t0,$0,1; b L2;
L1: add $t0,$0,$0; L2: sw $t0,a
```

(B)
```
lw $t0,a; beq $t0,$0,L1; sw $0,a; b L2;
L1: addi $t0,$0,1; sw $t0,a; L2:
```

(C)
```
lw $t0,a; sltiu $t0,$t0,1; sw $t0,a
```

(D)
```
lw $t0,a; addi $t1,$0,1; beq $t0,$0,L1;
addi $t1,$0,0; L1: sw $t1,a
```

(E) All of the above.

**Solution.** **E**. (A) jumps to L1 when $a \neq 0$ and stores 0, otherwise stores 1. (B) branches to L1 when $a = 0$ and stores 1, otherwise stores `$0`. (C) `sltiu` sets `$t0` $= 1$ iff $a < 1$ unsigned, i.e. iff $a = 0$. (D) presets 1 and overwrites it with 0 only when $a \neq 0$. Caveat: this assumes no branch delay slots (the assembler/simulator default). With delay slots exposed, the instruction after every branch always executes: in (A) the `add` after `b L2` would zero `$t0` before the store, and in (D) the `addi $t1,$0,0` would always run, so both would store 0. (B) and (C) would still be correct. □

## 3 Exam-Style Questions

Multi-part questions in the style of the in-class exercise. Solutions are in the last section.

**Exercise 3.1** (Encoding a loop). The following code starts at address `0x00400000`; the label `Done` is at `0x00400040`.

```
0x00400000  Loop: lw   $t1, 0($s0)
0x00400004        add  $t2, $t2, $t1
0x00400008        addi $s0, $s0, 4
0x0040000C        addi $t0, $t0, -1
0x00400010        bne  $t0, $zero, Loop
0x00400014        sw   $t2, 0($s1)
0x00400018        j    Done
```

(a) Give the format (R, I, J) and addressing mode of each instruction.

(b) Encode `lw` and `add` in hex.

(c) Encode both `addi` instructions in hex.

(d) Encode `bne` (compute the offset).

(e) Encode `sw` and `j`.

(f) In one sentence, what does the code compute?

**Exercise 3.2** (Decoding a function). A function is stored as these five words: `0x00854020`, `0x00084880`, `0x01264820`, `0x8D220000`, `0x03E00008`.

(a) Decode each word into a MIPS instruction.

(b) Give the format of each instruction.

(c) Write an equivalent C function (the arguments are in `$a0`–`$a2`).

(d) Is this a leaf procedure? Does it need a stack frame?

(e) Why does the code use `sll` by 2 before adding the base address?

**Exercise 3.3** (Procedures and the stack).

```
int sum3(int x, int y, int z) { return x + y + z; }
int g(int a) { return sum3(a, a + 1, a + 2) * 2; }
```

(a) Translate `sum3` into MIPS.

(b) Translate `g` into MIPS, following the calling convention.

(c) Why must `g` save `$ra` but `sum3` need not?

(d) Suppose `g` instead returned `sum3(…) * a`. What extra problem appears, and how do you solve it?

(e) Draw the stack contents of `g`'s frame while `sum3` is running (for the version in (d)).

(f) Write $D = A + B - C$ for a stack machine and for a load-store machine and count the instructions.

# 4 Practice Problems

**Exercise 4.1.** Decode `0x01095020` into a MIPS instruction.

**Exercise 4.2.** Decode `0x8E080004`.

**Exercise 4.3.** Encode `sub $s3,$s1,$s2` in binary and hex.

**Exercise 4.4.** Encode `sw $s1,-8($sp)`.

**Exercise 4.5.** `beq $t0,$zero,Exit` is at address `0x00400020` and `Exit` is at `0x00400040`. Find the offset field and the hex encoding. Then find the offset field of a `bne` at `0x0040003C` that jumps back to `0x00400020`.

**Exercise 4.6.** `j Target` is at `0x00400048`; `Target` = `0x00400100`. Give the 26-bit field and the full encoding.

**Exercise 4.7.** Load `0xABCD9234` into `$s0`. What goes wrong with `lui $s0,0xABCD; addi $s0,$s0,0x9234`?

**Exercise 4.8.** Name the addressing mode used for the operand/target in: `add $t0,$t1,$t2`; `addi $t0,$t0,4`; `lw $t0,8($sp)`; `beq $t0,$t1,L`; `j L`; `jr $ra`.

**Exercise 4.9.** Write $D = (A + B) \times C$ for a stack, accumulator, register-memory and load-store machine. Count instructions for each.

**Exercise 4.10.** `a[i] = a[i-1] + b;` with the base of `a` (word array) in `$s0`, `i` in `$s1`, `b` in `$s2`.

**Exercise 4.11.** `sum = 0; for (i = 0; i < n; i++) sum += A[i];` with `A` in `$a0`, `n` in `$a1`, result in `$v0`.

**Exercise 4.12.** Function `f` uses `$s0` and `$s1` and calls another function. Write its prologue and epilogue.

**Exercise 4.13.** The word `0x11223344` is stored at address `0x1000`. List the bytes at `0x1000`–`0x1003` on a big-endian and a little-endian machine. What does `lbu` from `0x1001` return on MIPS?

**Exercise 4.14.** What is the range of an `addi` immediate? How far can a `beq` reach? Where can a `j` jump?

**Exercise 4.15.** Expand into real instructions: `bgt $t0,$t1,L`, `not $s0,$s1`, `li $t0,-1`, `move $a0,$s2`.

## 5 Solutions to the Practice Problems

**Solution (4.1).** Hex to binary, split into fields:

`0000 0001 0000 1001 0101 0000 0010 0000` = `000000 01000 01001 01010 00000 100000`

R-format, rs = 8 (`$t0`), rt = 9 (`$t1`), rd = 10 (`$t2`), funct `0x20`: `add $t2,$t0,$t1`. □

**Solution (4.2).** `100011 10000 01000 0000000000000100`: opcode `0x23` (`lw`), rs = 16 (`$s0`), rt = 8 (`$t0`), imm = 4: `lw $t0,4($s0)`. □

**Solution (4.3).** rs = 17, rt = 18, rd = 19, funct `0x22`:

`000000 10001 10010 10011 00000 100010` = `0x02329822`

□

**Solution (4.4).** opcode `0x2b`, rs = `$sp` = 29, rt = `$s1` = 17, imm $-8$ = `0xFFF8`:

`101011 11101 10001 1111111111111000` = `0xAFB1FFF8`

□

**Solution (4.5).** Forward: $(0x40 - 0x24)/4 = 28/4 = 7$;

`000100 01000 00000 0000000000000111` = `0x11000007`

Backward: $(0x20 - 0x40)/4 = -8$, field `0xFFF8`. Remember that the reference point is PC+4, not the branch address. □

**Solution (4.6).** Field = $0x00400100 \gg 2$ = `0x0100040`. Encoding = $(2 \ll 26)$ | `0x0100040` = `0x08100040`. Upper 4 bits of PC+4 are 0, so the target is reachable. □

**Solution (4.7).** Correct: `lui $s0,0xABCD; ori $s0,$s0,0x9234`. With `addi` the immediate `0x9234` is sign-extended to `0xFFFF9234`, so the result is `0xABCD0000` + `0xFFFF9234` = `0xABCC9234` (wrong upper half). □

**Solution (4.8).** Register; immediate; displacement (base); PC-relative; pseudo-direct; register indirect. □

**Solution (4.9).** Stack: `PUSH A; PUSH B; ADD; PUSH C; MUL; POP D` (6). Accumulator: `LOAD A; ADD B; MUL C; STORE D` (4). Register-memory: `LOAD R1,A; ADD R1,R1,B; MUL R1,R1,C; STORE R1,D` (4). Load-store: `LOAD R1,A; LOAD R2,B; ADD R3,R1,R2; LOAD R4,C; MUL R5,R3,R4; STORE R5,D` (6). Load-store needs the most instructions, but each one is simple, fixed-length and easy to pipeline. □

**Solution (4.10).**

```
sll  $t0, $s1, 2      # i*4
add  $t0, $t0, $s0    # &a[i]
lw   $t1, -4($t0)     # a[i-1]
add  $t1, $t1, $s2    # + b
sw   $t1, 0($t0)      # a[i] = ...
```

□

**Solution (4.11).**

```
      add  $v0, $zero, $zero   # sum = 0
      add  $t0, $zero, $zero   # i = 0
Loop: slt  $t1, $t0, $a1       # i < n ?
      beq  $t1, $zero, Done
      sll  $t2, $t0, 2
      add  $t2, $t2, $a0       # &A[i]
      lw   $t3, 0($t2)
      add  $v0, $v0, $t3       # sum += A[i]
      addi $t0, $t0, 1         # i++
      j    Loop
Done:
```

□

**Solution (4.12).**

```
f:  addi $sp, $sp, -12
    sw   $ra, 8($sp)
    sw   $s0, 4($sp)
    sw   $s1, 0($sp)
    ...                  # body, including jal g
    lw   $s1, 0($sp)
    lw   $s0, 4($sp)
    lw   $ra, 8($sp)
    addi $sp, $sp, 12
    jr   $ra
```

□

**Solution (4.13).** Big-endian: `11 22 33 44`; little-endian: `44 33 22 11` (at `0x1000`..`0x1003`). MIPS is big-endian, so `lbu` from `0x1001` returns `0x22`. □

**Solution (4.14).** `addi`: $-32768 \dots 32767$. `beq`: $-2^{15} \dots 2^{15}-1$ words from PC+4, i.e. about ±128 KB. `j`: anywhere inside the same 256 MB region (same top 4 bits as PC+4); use `jr` for anything farther. □

**Solution (4.15).** `bgt`: `slt $at,$t1,$t0; bne $at,$zero,L`. `not`: `nor $s0,$s1,$zero`. `li $t0,-1`: `addi $t0,$zero,-1` (`ori` would zero-extend and give `0x0000FFFF`). `move`: `add $a0,$s2,$zero`. □

# 6 Solutions to the Exam-Style Questions

**Solution (3.1).**

(a) `lw`, `sw`: I, base+displacement; `add`: R, register; `addi`: I, immediate; `bne`: I, PC-relative; `j`: J, pseudo-direct.

(b) `lw $t1,0($s0)`: op `0x23`, rs = 16, rt = 9:

`100011 10000 01001 0000000000000000` = `0x8E090000`

`add $t2,$t2,$t1`: rs = 10, rt = 9, rd = 10, funct `0x20`:

`000000 01010 01001 01010 00000 100000` = `0x01495020`

(c) `addi $s0,$s0,4`:

`001000 10000 10000 0000000000000100` = `0x22100004`

`addi $t0,$t0,-1` ($-1$ = `0xFFFF`):

`001000 01000 01000 1111111111111111` = `0x2108FFFF`

(d) Offset = (`0x00400000` − `0x00400014`)/4 = $-20/4 = -5$ = `0xFFFB`; op `0x05`, rs = 8, rt = 0:

`000101 01000 00000 1111111111111011` = `0x1500FFFB`

(e) `sw $t2,0($s1)`: op `0x2b`, rs = 17, rt = 10:

`101011 10001 01010 0000000000000000` = `0xAE2A0000`

`j Done`: field = `0x00400040` ≫ 2 = `0x0100010`, word = `0x08000000` | `0x0100010` = `0x08100010`.

(f) It adds `$t0` consecutive words starting at address `$s0` into `$t2` (a do-while loop, so it assumes `$t0` ≥ 1), stores the sum at the address in `$s1`, and jumps to `Done`.

□

**Solution (3.2).**

(a)
```
000000 00100 00101 01000 00000 100000   add $t0,$a0,$a1
000000 00000 01000 01001 00010 000000   sll $t1,$t0,2
000000 01001 00110 01001 00000 100000   add $t1,$t1,$a2
100011 01001 00010 0000000000000000     lw $v0,0($t1)
000000 11111 00000 00000 00000 001000   jr $ra
```

(b) R, R, R, I, R.

(c) `int f(int a, int b, int *A) { return A[a + b]; }`

(d) Yes, it calls nothing, so `$ra` is never overwritten. It uses only `$t` registers (caller-saved), so it needs no stack frame.

(e) Array elements are 4-byte words: the byte offset of element $i$ is $4i$, and `sll` by 2 multiplies by 4.

□

**Solution (3.3).**

(a)
```
sum3: add  $v0, $a0, $a1
      add  $v0, $v0, $a2
      jr   $ra

g:    addi $sp, $sp, -4
      sw   $ra, 0($sp)       # g calls sum3, so save $ra
      addi $a1, $a0, 1       # y = a + 1
      addi $a2, $a0, 2       # z = a + 2   ($a0 = a already)
```

```
        jal  sum3
        add  $v0, $v0, $v0     # * 2
        lw   $ra, 0($sp)
        addi $sp, $sp, 4
        jr   $ra
```

(b) `jal sum3` overwrites `$ra`; without saving it, `g` could not return to its own caller. `sum3` is a leaf, so its `$ra` stays intact.

(d) `a` lives in `$a0`, which is not preserved across the call. Keep it in a callee-saved register (save that register first):

```
g:      addi $sp, $sp, -8
        sw   $ra, 4($sp)
        sw   $s0, 0($sp)
        add  $s0, $a0, $zero   # keep a across the call
        addi $a1, $a0, 1
        addi $a2, $a0, 2
        jal  sum3
        mul  $v0, $v0, $s0     # * a
        lw   $s0, 0($sp)
        lw   $ra, 4($sp)
        addi $sp, $sp, 8
        jr   $ra
```

(e) From high to low addresses: [caller's frame] → saved `$ra` (at `$sp+4`) → saved `$s0` (at `$sp+0`, where `$sp` points). `sum3` adds nothing.

(f) Stack: `PUSH A; PUSH B; ADD; PUSH C; SUB; POP D` (6). Load-store: `LOAD R1,A; LOAD R2,B; ADD R3,R1,R2; LOAD R4,C; SUB R5,R3,R4; STORE R5,D` (6). Same count, but the load-store version can overlap independent loads and keeps values in registers for reuse.

□